How Hardware Gets Hacked (Part 7)

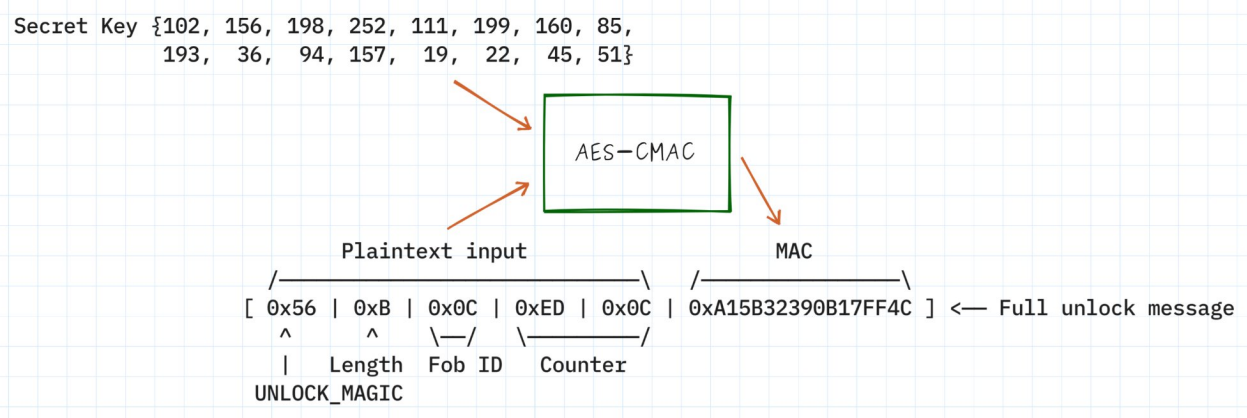
Freshness and Randomness

by Nathan Jones

Introduction

In our last two articles, we discovered (or perhaps you knew all along) that our method of unlocking the car was highly insecure: sending `[ 0x56 | 0x6 | “unlock” ]` to unlock any car was susceptible to several attacks that could be broadly classified as “replay attacks”. Our solution was to enforce authenticity of each message by requiring a fob to compute the “message authentication code” (MAC) of a rolling counter using AES-CMAC. The only way an attacker could correctly calculate the MAC for a given counter value was if they knew the secret key (we can assume they don’t) and they couldn’t replay any

messages since the rolling counter ensured that every message was slightly different than any before it.

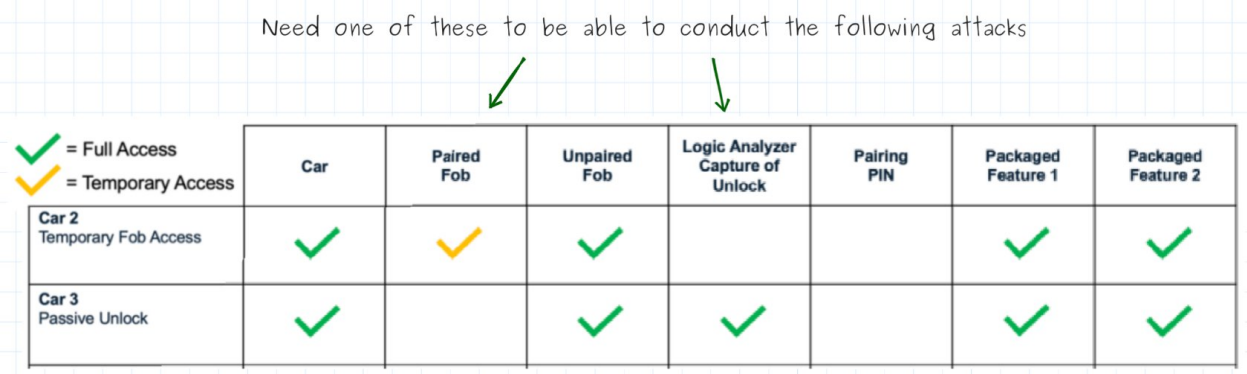


This defense may feel pretty sophisticated (we're using real cryptographic algorithms from real cryptographic libraries!) and, indeed, it shuts down many *primitive* attacks. And yet, attackers are more sophisticated still and even this defense is not unassailable. In this article, we'll see how a determined attacker can *still* successfully conduct a replay attack on our car by exploiting either the lack of "freshness" in our unlock messages or the physical vulnerabilities of the device itself. Our new defense will also require us to discuss the nature of randomness itself and how to generate it on a device (our MCU) that was designed to be highly deterministic.

Sophisticated replay attacks

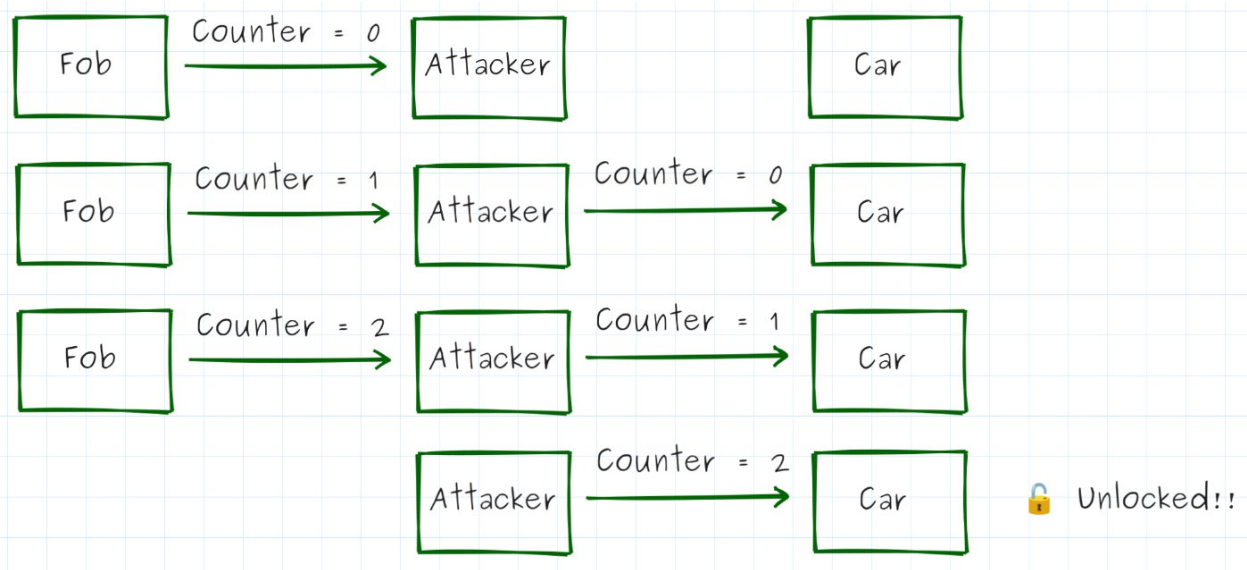
The attacks we addressed in the last article were "simple" replay attacks: see an unlock message from a fob and copy it, hoping the duplicate message will cause the car to unlock. Although our rolling counter + MAC scheme shut down this type of attack, there are three different, more complicated forms of replay attacks that can still allow an attacker to replay an old (a.k.a. "stale") message and successfully unlock a car.

All of these attacks rely on having a valid unlock message to begin with, and so they target specifically **Cars #2 and #3** from the competition. Cars #1 and 4 are safe from these attacks, since those scenarios didn't come with paired fobs and we don't yet have any way to capture a valid unlock message (which each attack below relies on) without a paired fob in hand.



Attack #3a: "RollJam"

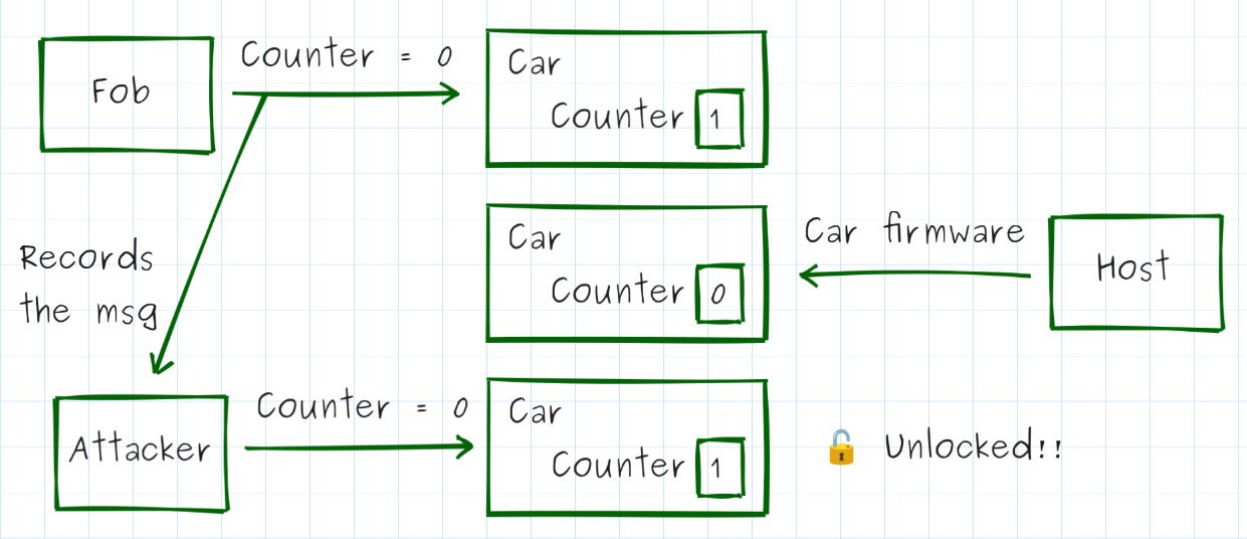
"RollJam" is an attack that was discovered by Samy Kamkar and first presented by him at DEF CON 23. A RollJam attack works by intercepting every unlock message from a key fob and always storing the most recent message, passing the next-most-recent on to the car.



As long as the key fob doesn't unlock the car when the attacker's device isn't listening, the attacker has a valid unlock message they can use in the near future to unlock the car!

Attack #3b: Forced rollback

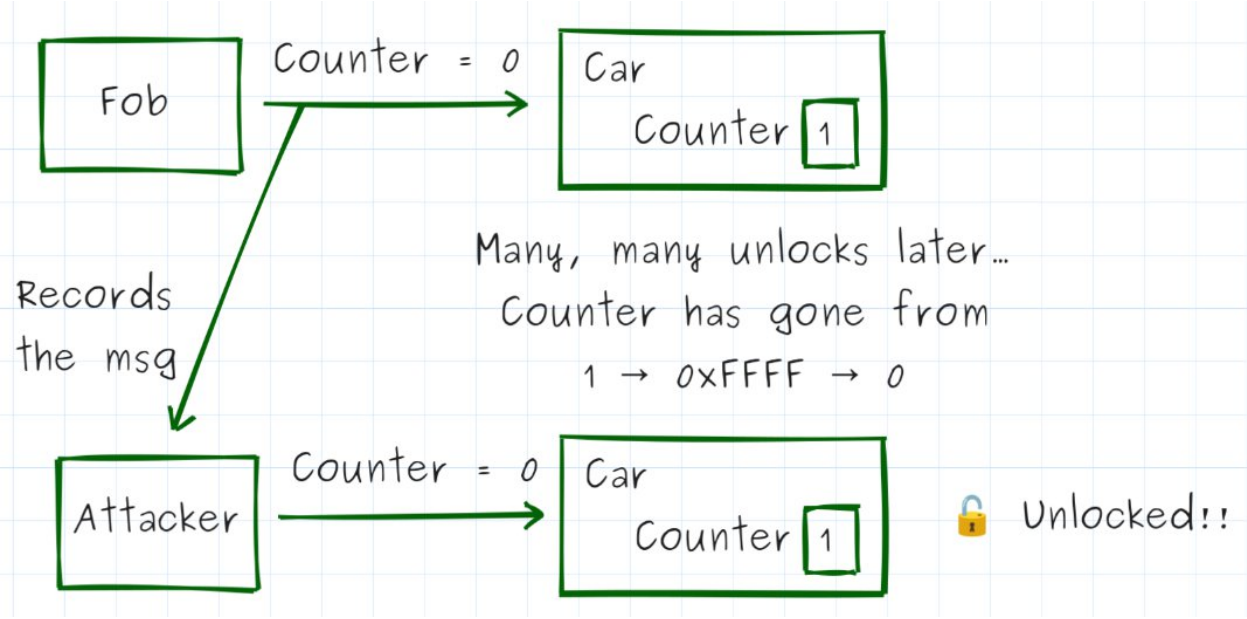
A key security property of our defense is that the rolling counter stored in the car is **monotonically increasing** (it never decreases). Every unlock message is only good once because after that the car has moved on to the next higher number. However, since we have ability to re-flash the car's firmware at will, we can effectively reset the car's counter back to zero whenever we want!



To conduct this attack, we first capture an unlock message from the paired fob. Then, we re-flash the opposing team's firmware onto the car, which resets any internal counters in memory to zero. (If the counter values persist re-flash cycles, we can still conduct a mass erase of the device and then re-flash.) Then we can send the "fresh" car our captured unlock message to unlock it.

Attack #3c: Forced rollover

This attack is a variation on "forced rollback", based on the fact that my statement above ("Every unlock message is only good once because after that the car has moved on to the next higher number") is not *entirely* true. In truth, every counter rolls over to zero at some point!



In this attack, as in the last one, we capture the first unlock message a paired fob sends to the car. Then we trigger a whole LOT of unlocks (up to 65,536 if the counters are stored as `uint16_t`; 4,294,967,296 if the counters are stored as `uint32_t`, etc), causing the counter to take on a high enough value that the original message (the one we captured at the beginning) is within the range of accepted values. We can then use that message to unlock the car!

`/******[🤔 What feature does "Forced Rollback" exploit?]**********`

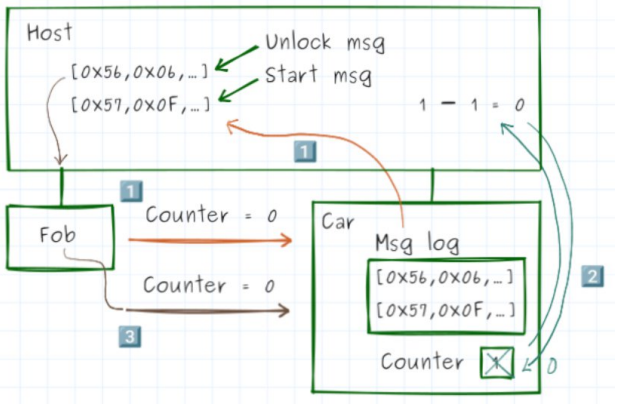
Why does the forced rollback attack (attack #3b) succeed even when AES-CMAC is used? Which implementation choice does it exploit? (Answer[i])

`**********************************************************************/`

Adding these attacks to the security tests

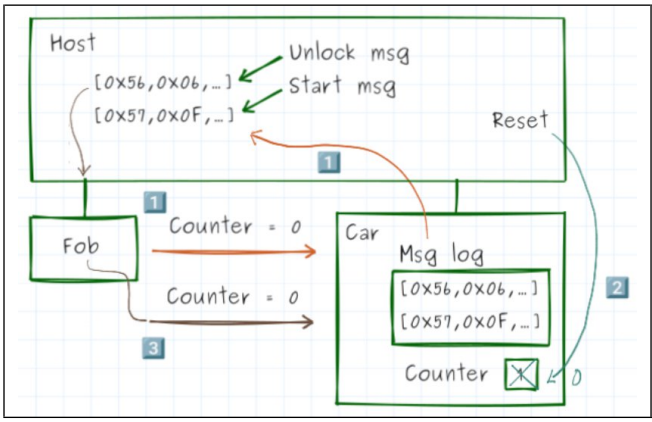
Adding these attacks to the current suite of security tests required the implementation of two new test commands (`getFlashData` and `setFlashData` for the car, which load or store the array of fob counters) and for the `reset` command on the car to be extended to also clear the array of fob counters. Then the tests could be implemented as described below.

RollJam test



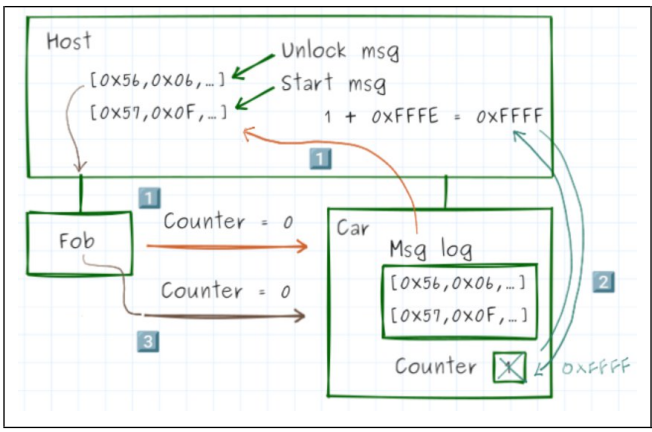
1. Test triggers unlock and reads car message log to get the unlock message that was sent by the fob.
2. Test gets the car's flash data, decrements the counter for this fob, and sets the new data (this simulates the car having never heard the last unlock message, since the attacker "intercepted" it)
3. Test forces fob to send out the previous unlock message

Forced rollback test



1. Test triggers unlock and reads car message log to get the unlock message that was sent by the fob.
2. Test resets the car (which zeros out the array of fob counters)
3. Test forces fob to send out the previous unlock message

Forced rollover test



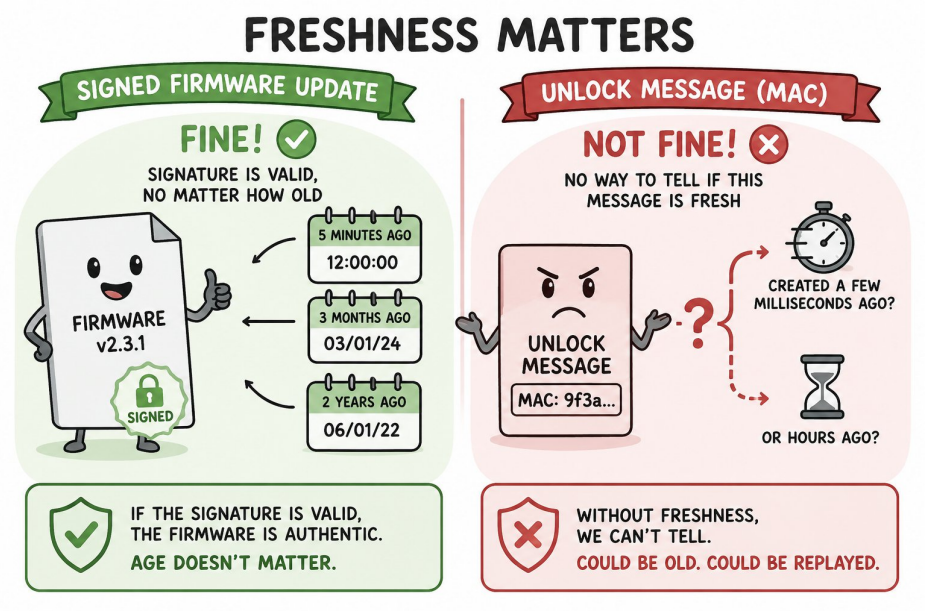
1. Test triggers unlock and reads car message log to get the unlock message that was sent by the fob.
2. Test gets the car's flash data, sets the counter for this fob to `UINT16_MAX`, and sets the new data (this simulates the car having received `UINT16_MAX - 1` unlock requests)
3. Test forces fob to send out the stored unlock message

The fundamental problem

The fundamental problem is that our current scheme only proves that the received message was generated by an authenticated fob *at some point in the past*. It has to no way of verifying that the message was generated *right now* by an authenticated fob.

It turns out that proving authenticity is slightly different for **static artifacts** versus **live messages** (particularly messages that contain or are associated with commands like "unlock"). A example of a static artifact is a signed firmware update. It doesn't matter *when* the firmware update was created, only that it has the valid signature from the manufacturer when a device, at any point in the future, uses it to update itself. For these, a simple MAC (or a digital signature from a public-key cryptographic system like RSA) is completely sufficient. As identified, this is insufficient for our purposes, however, since we

also need to know that a message has been authenticated *right now*; we need to know that the message is "fresh".



Proving "freshness" would be a difficult challenge for a device that didn't support two-way communication, as early key fobs and garage door openers didn't (which is one reason why rolling counters are still so prevalent in many devices built around one-way communication). For our system, though, we *do* have the ability for a car and a fob to communicate back and forth and it's this feature that critically enables our defense.

We can prove that an authenticated fob is requesting to unlock a car by asking it to **respond to a challenge** from the car, in real-time.

1. A fob says, "Unlock?"
2. The car says, "Prove it's you!"
3. The fob provides a response that proves it knows the car's secret (key) and which could only have been generated after the conversation had started

This is like getting a phone call from a loved one with some shocking news. If you were at all worried that someone had merely recorded your loved one's voice and was playing it back to you out of order to create this suspicious message (Who's paranoid? You? Nah...), one thing you might do is ask them to repeat back something you just said to prove they weren't a recording. We're effectively doing the same thing here.

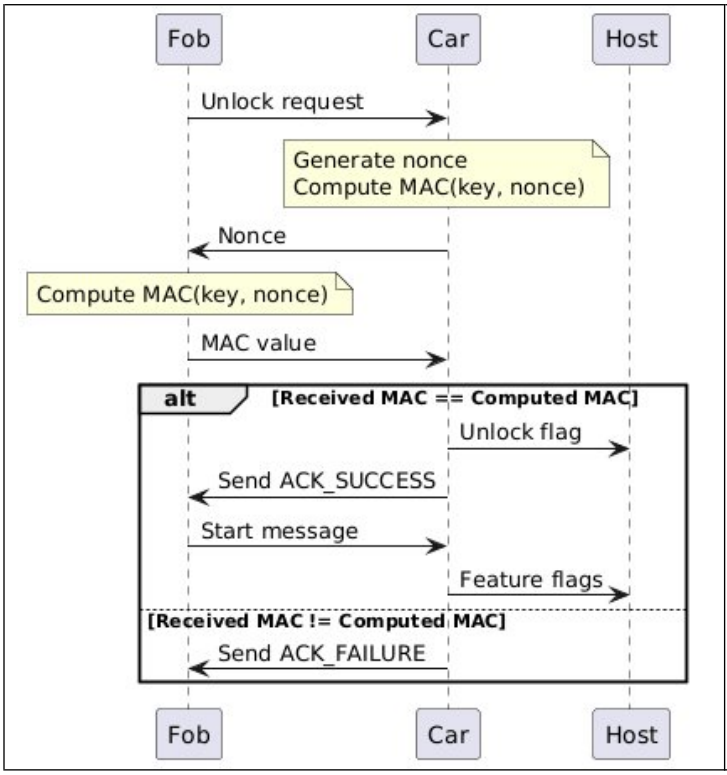
`/******[⍰ Say my name!]**************************`

Proving that a message was generated *right now* — not at some point in the past — is called proving its ____. (Answer[ii])

**********/

Defense #3: Challenge-Response

We already know how to securely prove that a fob knows a car's secret: compute a MAC over a known value. In this case, though, we want that value to come from the car and we want it to be a randomly generated value. This value is called a "nonce" ("**n**umber used only **once**"). As long as its sufficiently random, no attacker could possibly predict what any future "challenge" value could be and neither could they compute the correct MAC value without knowing the car's key. The exchange will look like this:



1. The fob sends an unlock request
2. The car generates a random value (nonce) and sends it to the fob, then computes the MAC value of the nonce using its key
3. The fob computes the MAC value of the nonce and sends it back
4. The car compares the received MAC value to its own computed MAC value

This should prevent any of the modified replay attacks above, since even if the attacker captures a fob's transmissions, they're only valid for a specific nonce which the car won't ever issue again.

It is critical, then, that the car be able to **generate pseudo-random nonces** (an attacker can't correctly guess any future values) and for the **nonces to not repeat** over some long period of time (which is a function of both the nonce width [should be 32-64 bits] and the PRNG construction). It's a subtly difficult challenge to write a correct pseudo-random number generator (PRNG), so let's talk about how to do that!

```
/******[? Why don't we use a 16-bit nonce?]**********
```

If nonces were only 16 bits wide, how many unlock events would need to happen before a captured challenge-response pair could probabilistically be reused? At one unlock attempt per second, how long would that take an attacker to force? (Answer[iii])

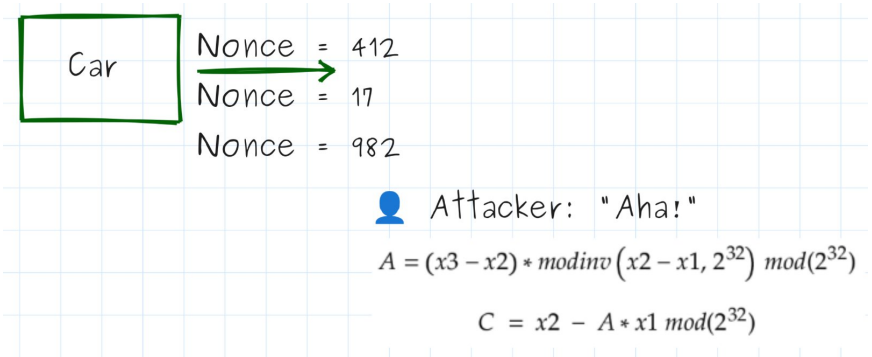
```
**************************************************/
```

Generating nonces

A naive way to generate nonces (as we learned in Part 5) is to use a linear congruential generator, i.e. `rand()`:

```
// Linear congruential generator (LCG)
#define A     1664525UL
#define C  1013904223UL
static uint32_t state = 0;
void seed(uint32_t val){ state = val; }
uint32_t rand(void){ return state = (uint32_t)(A*state)+C; }
```

Even if A, C, and state are kept secret (i.e. generated randomly as part of the build process), though, it only takes 3 consecutive outputs to determine them and then have the ability to guess all future values.



If an attacker could correctly guess the value of a future nonce, they could feed it to a fob and record the fob's response, thus having a valid response message when the car next issues the nonce!

What's needed is a true **one-way function**, something with no mathematical way for a person who sees any number of nonces to ever deduce the inputs that went into the function.

"What's that you say? We've *already discovered* a function with that property?"

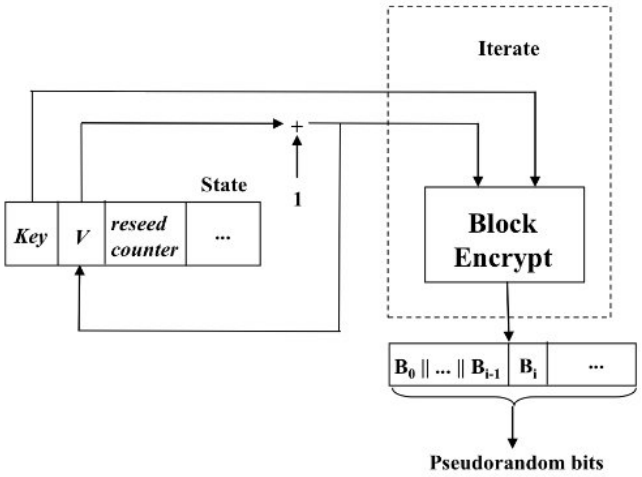
Right you are! Functions that compute MACs (like AES-CMAC, which we're currently using) are themselves one-way functions. We can compute the MAC of a piece of state to generate nonces that an attacker can't distinguish from noise.

```
static uint8_t prng_key[16] = {...};  // Secret, made at build-time
uint32_t rand(void){ return /* AES_CMAC(key, piece_of_state) */ ; }
```

The "state" over which the MAC is being computed could be

- a counter:
  ```
  static uint32_t counter = 0;
  uint32_t rand(void)
  {
    // return upper 2 bytes of AES_CMAC(prng_key, counter++);
  }
  ```
- or the output of the PRNG itself:
  ```
  static uint8_t state[16] = {0};
  uint32_t rand(void)
  {
    // Update state variable: state = AES_CMAC(prng_key, state)
    // return upper 2 bytes of state
  }
  ```

In fact, the counter-based PRNG construction above is essentially how NIST (National Institute of Standards and Technology) recommends implementing CTR-DRBG ("counter-based deterministic random bit generator") in SP 800-80A. Notice, below, how the "pseudorandom bits" from CTR-DRBG are produced by encrypting a key with an incremented counter ("V").



The full CTR-DRBG implementation is a little more involved, with ways to update the key and counter after each new pseudo-random number and to reseed the PRNG, but not by much. Popular cryptographic libraries like wolfSSL or mbedTLS have implementations like this as part of their code but those libraries are still just a little too heavyweight for me at this point. So instead of using them, I made a simple implementation of CTR-DRBG that I then validated against the NIST CAVS test vectors for CTR_DRBG (AES-128, no derivation function). You can see the implementation here, and you can run the tests that compare its output to the NIST test vectors by running `pytest testing/test_ctr_drbg_cavs.py`.

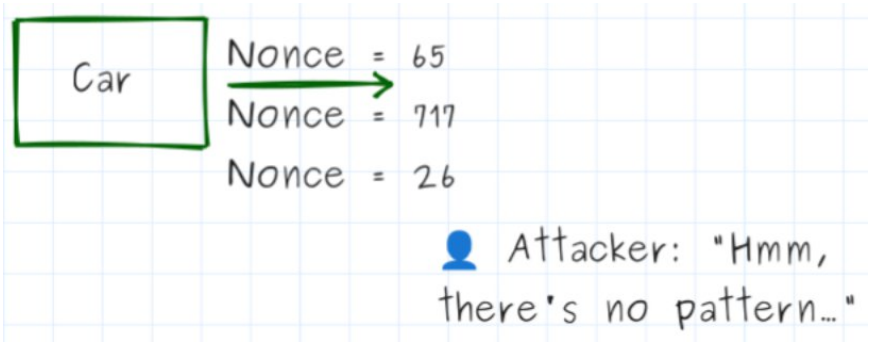
```
/******[⚠ The PRNG key is NOT the same as the unlock key]**********
```

Notice that we're using a second key, now, we're *not* just reusing the unlock key! That would be like having the same key to unlock your house *and* your shed *and* your office *and* your storage unit down the road: the loss or compromise of that one key would give an attacker access to *all* of those things. Instead, it's good security practice to use unique keys for each unique purpose (called "key separation"). If generating and storing those keys becomes a problem, one solution is to mathematically derive each unique key from a single stored key, for example:

```
static uint8_t device_key[16] = {...};  // Secret, made at build-time
static uint8_t unlock_key[16] = AES_CMAC(device_key, "unlock");
static uint8_t prng_key[16] = AES_CMAC(device_key, "prng");

**********************************************************************/
```

Now no matter how many nonces an attacker sees, they can't feasibly predict any future nonce values unless they also happen to know the PRNG key.



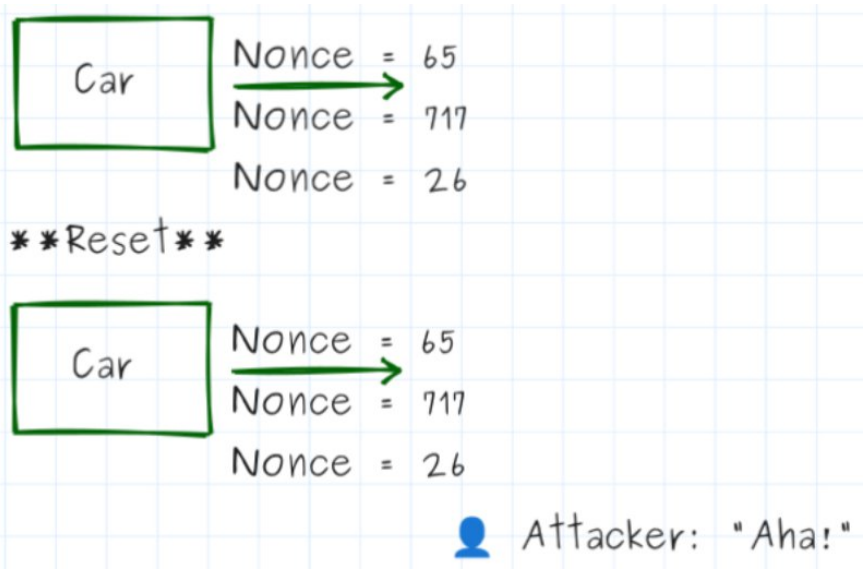

```
/******[? PRNG key compromise]********************
```

If the PRNG key were recovered from a car (e.g., via a debug port attack from a previous article), but the attacker doesn't have any existing captured challenge-response pairs, what could they do? Would they be able to generate valid unlock messages on demand, or do they still need something from the fob? (Answer[iv])

**********/

Seeding the PRNG

But there's one last problem to solve! Since our attacker still has control of the physical devices, they could potentially reset the car whenever they wanted. And since our PRNG currently always starts from a known state, they would see the *same sequence of "random" numbers* after each start-up!



So the last piece to this puzzle is to find a way to randomly **seed** the PRNG at each start-up so that it begins its pseudo-random sequence with a different starting value each time.

Yes, yes we do. Thankfully, though, there *are* ways to kickstart this process by accumulating enough "randomness" from other parts of the MCU. In fact, we *could* actually use the process below to exclusively generate all of our pseudo-random numbers; the biggest downside is that it takes a lot longer to do that then to simply compute a MAC, so if

we care about being able to get a random number quickly we'll still use this process just to *start* the PRNG and then use the PRNG to generate all future values.

Randomness can potentially be found on an MCU in a number of places:

- Unique **per-device ID** that was burned into the MCU's ROM during manufacturing
- **SRAM startup values**: On a "cold boot", the exact values in RAM depend on manufacturing variation and recent thermal history (which is also the cause of the classic "uninitialized RAM" bug)
- HSI/LSI **clock jitter**: If the high- and low-speed internal clocks come from different sources, then they'll drift in different ways. The number of HSI clock ticks between each period of the LSI clock source thus varies from period to period.
- ADC measurement of $\mathbf{V_{cc}}$ **relative to** $\mathbf{V_{refint}/V_{bandgap}}$: Varies with manufacturing and thermal differences in V_{refint} and with supply noise on V_{cc}
- On-chip **temperature sensor** (upper bits will be relatively constant, but the LSBs will appear pseudo-random)
- ADC measurement of a **floating pin** (not tied to any voltage source)

Each one varies in how often they change values (if at all), how many bits of randomness we can expect from them, and if they are correlated with other source of randomness. The table below summarizes these qualities for the sources that were listed above.

Source of randomness	Frequency of change	Bits of randomness	Notes
Device ID	Once during manufacturing	Typically 32-96	
RAM start-up values	Once per "cold boot"	~10-20% of RAM bits	Invalid if device was simply reset, not power-cycled
Clock jitter	Once per LSI clock period	2-4	
ADC: V_{cc}	Once per microsecond	1-3	Can be partially subverted by an attacker with physical access (use very low-noise power supply)
ADC: temp sensor		1-2	
ADC: floating pin		1-2	Easily defeated by an attacker with physical access (tie pin to Vcc/GND)

The top two sources are good for creating per-device random numbers but not for our purposes of generating new random numbers each time the devices restarts (device ID clearly doesn't change from boot to boot and the RAM values are only valid if the device was actually power-cycled, which is not easily verified by the device itself). The rest *can* be used for this purpose and they *can* be used multiple times to accumulate more and more randomness, provided subsequent readings happen no faster than their "frequency of change".

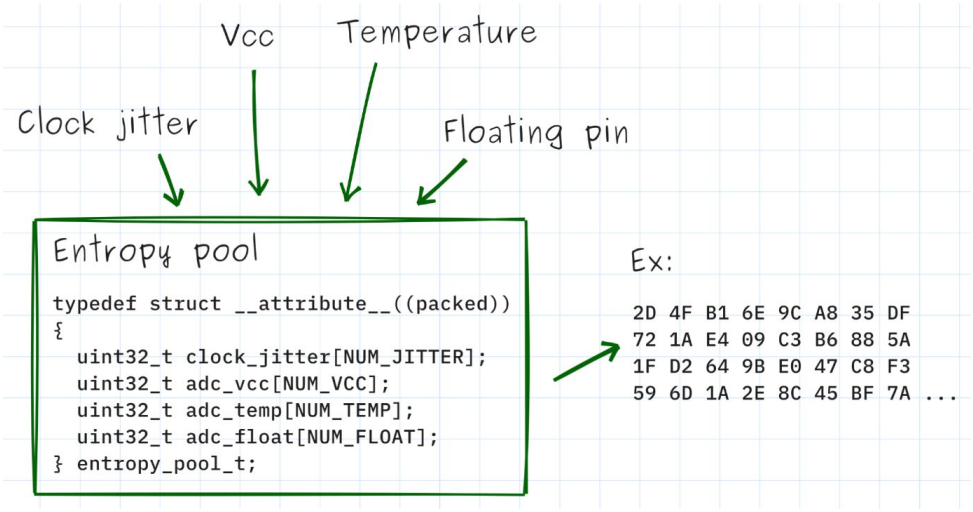
The only way to know with any confidence how many bits of randomness you can get from a source on your MCU is to test it, ideally using NIST's SP800-90B toolkit. This software takes a large number of samples from your entropy source and conducts a number of tests on it to determine how much randomness it exhibits. In commit #cdefe56, I implemented a test to capture these samples from the STM32 and TM4C and to run two of the assessments in that toolkit: the **initial entropy estimate** and the **restart test**. The table below shows how many bits of randomness I measured from each of the sources that were available on those two devices.

Source of entropy	**STM32**	**TM4C**
V_{cc}	~~0.350~~	N/A
Temperature sensor	0.874	1.251
Floating ADC pin	2.821	1.540
HSI/LSI clock jitter	1.809	N/A

Two of the ADC measurements on the STM32 actually have *less* entropy than the lower-bound in my table above, so it's a good thing we measured it instead of assuming that they each provided a full bit of entropy. And, actually, measuring V_{cc} on the STM32 failed the restart test since too many of the values measured immediately after a reset were identical (788 out of 1000!), so we can't really use that as a source of entropy in this case. (This does make sense when you think about it, since V_{cc} is typically *designed* to be as stable as possible.)

Additionally, my test performs a pairwise comparison of each pair of entropy sources to see if there is any correlation between them (not part of the NIST toolkit). This could happen if, for instance, the dominant source of randomness in each of the ADC measurements was internal ADC noise, which would show up regardless of what was actually being measured by the ADC. Those sources would *not* give us additional entropy if each one was essentially a result of the same underlying noise. In each case I tested the correlation was small (almost small enough to ignore), but non-zero.

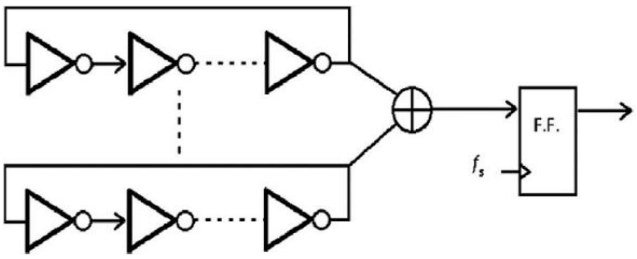
To generate the random seed to start our PRNG, first we accumulate enough of these semi-random sources in an array or struct to have 128 bits of randomness (as required by the standard for CTR-DRBG). This is additive across each of our values (so we could use, for instance, 128 ADC readings or 64 clock jitter readings or 64 ADC readings and 32 clock jitter readings, etc), minus any correlation between each of the sources. More samples are better, since we're not sure exactly how much randomness each measurement is providing and it's best to incorporate as many sources as possible, to avoid any correlation in our readings that might reduce the amount of randomness we're actually accumulating. We also want enough bits of randomness that an attacker who was repeatedly resetting a device wouldn't likely cause the car to start with the same seed value, since this would enable the replay attacks again. If we were using the PRNG to generate cryptographic keys or for key derivation, however, we'd want as many bits of randomness as there were bits in the key itself (typically 128 or 256).



The accumulation of all these sources of randomness is called an **entropy pool** and the process of acquiring it is called **entropy accumulation**.

`/******[▶ TRNGs and ring oscillators ]**********************`

You know what would be even better than having to make our own randomness? Getting truly random numbers for free! Many MCUs with security-conscious peripherals will include a "true random number generator" (TRNG) that can produce random numbers on-demand much better than we could do with our PRNG. These TRNGs typically use two or more ring oscillators (which is an odd-numbered chain of inverters that self-oscillates at a frequency determined by its gate propagation delay) that are XORed together to create a truly random stream of 1s and 0s.



https://www.researchgate.net/profile/Bharat-Meitei/publication/353965849/figure/fig1/AS:1137776447754242@1648278452730/General-architecture-diagram-for-TRNG-based-on-Ring-Oscillator.ppm

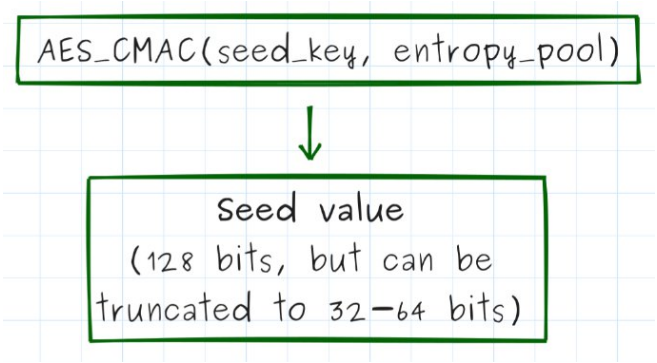
You could always construct your own ring oscillator out of discrete components and read its output from your MCU, though this is susceptible to physical tampering from an attacker. Other options include an avalanche noise circuit, Johnson-Nyquist noise amplifier, or a dedicated TRNG chip such as the Microchip ATECC608 or Maxim DS28C36.

**********/

Lastly, we want to condition our entropy pool by computing its MAC value. Applying AES-CMAC to our entropy pool ***does not*** increase the entropy, but it still serves several useful purposes:

- It condenses our large data struct to a manageable 128 bits
- It **whitens** our final value (spreads out the distribution to look more random)
- It causes wild changes in the output for any one single bit flip of input (a property known as "**avalanche**"; AES-CMAC actually guarantees that, on average, 50% of the output bits will change if even one bit of its input were to change)

None of this *adds* any randomness to our entropy pool, as mentioned, but it does help the final value look indistinguishable from noise to an attacker. Note that we'll need to use another unique key ("seed_key"), separate from the `prng_key` and the `unlock_key`, to do the MAC calculation.



Here are the links to see the implementations for seeding our PRNG using the STM32, TM4C, and our simulation environment.

`/******[? Maximize your retention!]**********************`

Which of the following best describes what an entropy pool is?

A. A cache of previously generated nonces
B. The internal state of the PRNG after seeding
C. Accumulated semi-random data from multiple hardware sources
D. A lookup table mapping seeds to PRNG outputs

(Answer[v])

`**************************************************/`

"But does it work??"

Yes! Our security tests now pass, though the "RollJam" and "Forced rollover" attacks are being skipped since they don't make any sense in the context of our challenge-response system.

```
> pytest testing/security_tests.py
============================================================================== test session starts ==========
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 9 items

testing/security_tests.py ..s.s..x.

============================================================================ 6 passed, 2 skipped, 1 xfailed in 34.84s =
```

(The one "xFailed" above [and below] is the result of the simulation environment not producing enough bits of entropy, but the simulation only needs to "defend" against the security tests while the hardware will need to defend itself against real-life attackers, so this is not bothersome to me.)

I also added tests that:

1. check the amount of entropy being generated by our seed function (add `-v -s` to the `pytest` command to make sure the messages below are printed and not suppressed),

   ```
   Seed entropy estimate:  39.4 bits  (90% of 44-bit ceiling at N=50)  [GOOD  (source looks uniform)]
   Per-byte: ['2.5', '2.5', '2.5', '2.5', '2.5', '2.3', '2.5', '2.5', '2.5', '2.5', '2.5', '2.5', '2.5', '2.5', '2.5', '2.5']
   Note: MCV ceiling grows with N. Run with --n-samples=5000 for a meaningful estimate of an 8-bit/byte source.
   PASSED

   ============================================================================ 1 passed in 67.09s (0:01:07) ========
   ```

   (Note: I didn't learn until after I wrote this test that it's not really a correct estimation of our seed function's entropy. If you check out the commit associated

with this article, commit #8c758ac, you should probably ignore the results of this test. It wasn't until commit #cdefe56 that I implemented the NIST SP800-90B toolkit, mentioned above, that accurately determines the entropy of our sources and which was used to get the entropy numbers in the table a few sections back. That test takes a *long* time to run (1-2 hours, depending on your host computer) and produces results that look like the image below.

```
Collecting 1000000 interleaved rows from 2 source(s) (temp:2B, float_pin:2B)...
  [##############################] 1000000/1000000

Saved capture: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/
Saved metadata: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code
[temp] track=non_iid iid_permutation=False assessed_min_entropy=2.040 bits/byte (25.5% of 8-bit ceiling)
[float_pin] track=non_iid iid_permutation=False assessed_min_entropy=1.797 bits/byte (22.5% of 8-bit ceiling)

Pairwise source comparison:
  temp <-> float_pin: pearson_r=+0.2635, mutual_info~=0.2694 bits
```

2. confirm that our implementation of CTR-DRBG matches the NIST test vectors (already mentioned),

```
> pytest testing/test_ctr_drbg_cavs.py
============================================================================== test session starts ==============
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 29 items

testing/test_ctr_drbg_cavs.py .............................

============================================================================== 29 passed in 0.34s ===============
```

3. and that the actual nonces being generated aren't trivially similar to one another.

```
> pytest testing/security_tests.py::TestNonceRandomness
============================================================================== test session starts ==============
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 4 items

testing/security_tests.py ..x.

============================================================================== 3 passed, 1 xfailed in 32.37s ======
```

Alternatives

There are a few alternatives worth mentioning as being viable solutions that could work under different conditions or that help solve our problem in a different way.

The first is to use a **"time-based one time password"** (TOTP), which would require the fob and car to be able to keep synchronized time (either with a very stable RTC or a network connection to get network time). In that scenario, the fob would send a timestamp alongside its unlock message. If the car could verify that the MAC value over the message (including the time value) matched the one it had computed with its secret key, *and* that the time value was within some small window of its current time, then it would unlock. A correct MAC value would ensure authenticity and a recent timestamp would ensure freshness.

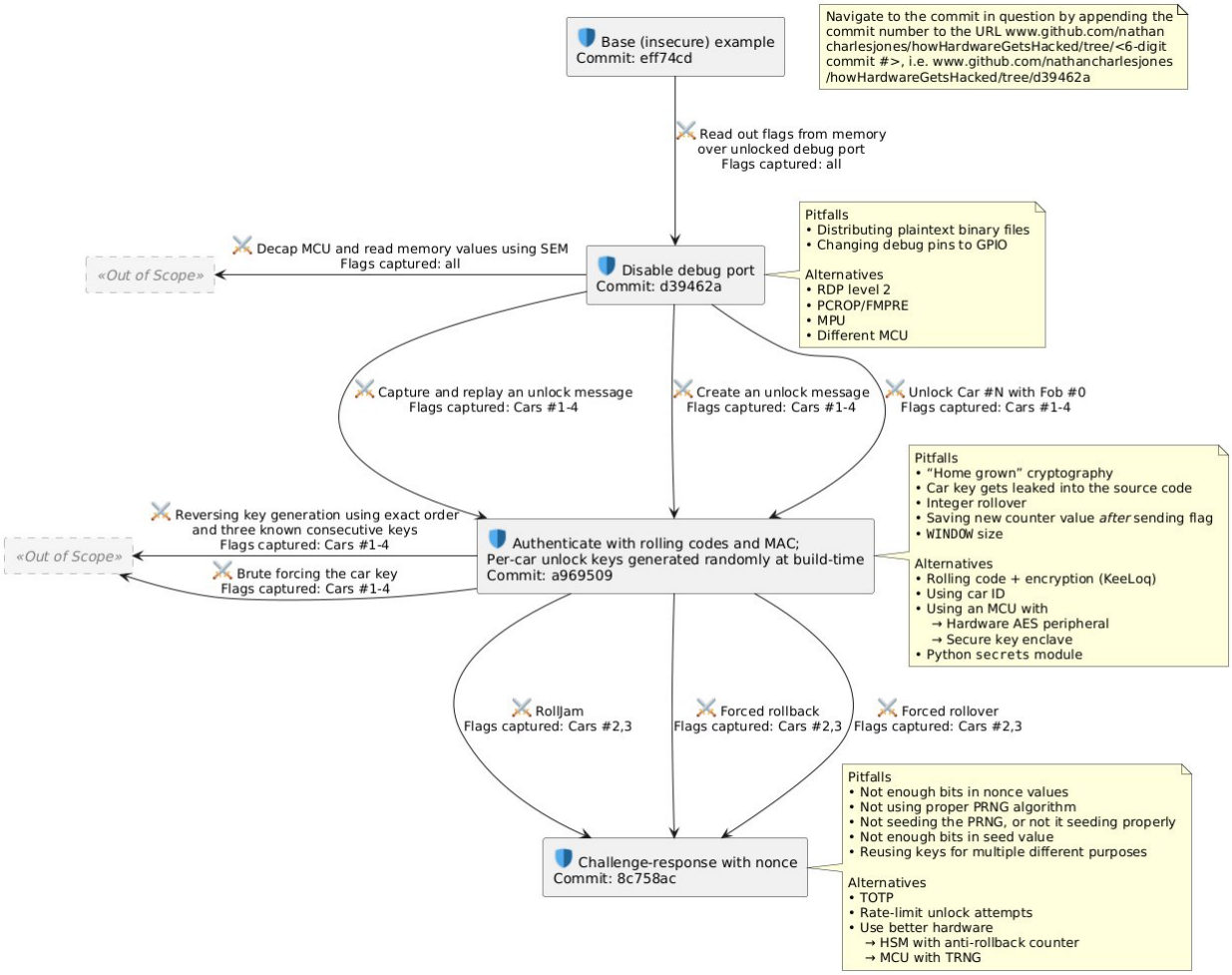
The second is to **rate-limit** the number of unlock attempts that can be made in a given amount of time. If this were limited to, say, 3 unlock attempts per second (more than

sufficient for a real person who's trying to unlock their car), then this could make attack #3c take a little over 6 hours to execute, which could be annoying enough to deter an attacker. The implementation of this feature would have to account for the fact that an attacker can re-flash a device, though. By a similar token, attack #3c could be made more difficult by **increasing the maximum size of a counter** from 16-bits to 32-bits or more.

The third is to use a **hardware security module** (HSM) with built-in **anti-rollback counters**, such as the Microchip ATECC608B. These are counters that can only ever increase, never decrease. If used to store the rolling counters sent by the fobs, then these modules could prevent attack #3b.

The fourth and final alternative is to use an MCU with a **true random number generator** (TRNG), such as an STM32F407 or nRF52840 (housed on the NINA-B3 transceiver module). Having a TRNG would obviate the need for our somewhat convoluted process of seeding and running a PRNG, and it will usually have been tested to a much higher standard to guarantee randomness. External circuits can also be added to a device to produce randomness, but being external to the MCU, these circuit are also subject to attack.

Updated threat model



Conclusion

Although a rolling code + MAC may have *felt* very secure at the end of the last article, it ultimately fell victim to a few complex replay attacks, like RollJam and "forced rollback", that exploited the fact that the car couldn't tell how *recently* an unlock request has been made, even if it could verify that the message still originated from a paired fob.

The solution was to enforce "freshness" via a challenge and response:

- A fob requests an unlock
- The car responds with a random "nonce" (number used only once)
- The fob computes the MAC of the nonce and sends it back to the car
- The car checks if the fob's MAC matches the MAC it calculated for the same nonce

For this to work, it's critical that a car be able to generate random numbers that an attacker can't predict and that don't repeat over a sufficiently long period of time. To generate numbers that an attacker can't exploit, the car needs to use a mathematically verified, one-way function that's been seeded properly using a sufficient number of entropy sources that have been conditioned properly.

If you've made it this far, thanks for reading and happy hacking!

---

[i] The "forced rollback" attack exploits the fact that counters are stored in flash memory, which the attacker can erase and re-program.

[ii] Freshness

[iii] If there are only 2^{16} possible nonce values, then an attacker would need, at most, 2^{16} unlock events to see a repeated value, or just $N \cdot ln(2) = 2^{16} \cdot 0.693 = 45416$ events for a 50% chance of seeing a repeated value. At one unlock event per second, that's only 12.6 hours for 50% chances and only 18.2 hours for all 2^{16} values (essentially a 100% chance).

[iv] The attacker can correctly guess all future nonces, though the exact attack depends on whether the PRNG is using a counter or the output of the PRNG itself. If the latter, the attacker immediately has access to all future nonce values (since they have the last value and the key). If the former, the attacker only needs to compute each value of the PRNG, beginning from `counter = 0`, until they reach the one last seen.

[v] C